Employees Database with SQL Constraints

MySQL Assignment 1 — DDL Commands & Constraints

Module 3 · AI-Driven Data Analytics

Field	Detail
Author	Sharu Latha B
Course	Data Analytics (DA) — Module 3
Tool	MySQL Workbench 8.0
Scripts	Employee_DB_Assignment.sql, Employee_DB_Queries.sql

1. Project Overview

- Create a relational database for managing employee information.
- Apply SQL DDL commands and constraints that enforce rules and prevent invalid data.
- Demonstrate the importance of schema design in real-world applications.

2. Description

The Employees Database is built using SQL Data Definition Language (DDL) commands and includes three main tables:

- Departments — stores department details.
- Locations — stores office location details.
- Employees — stores employee records with references to departments and locations.

3. Objectives

- Create tables for Employees, Departments, and Locations.
- Practice the five core DDL commands: CREATE, ALTER, RENAME, TRUNCATE, DROP.
- Apply constraints: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT.
- Insert sample data and test constraints with valid and invalid inputs.
- Explore the data with INNER JOIN / LEFT JOIN and analytical queries.

4. Part A — DDL Commands

Task 1 — Database & Table Creation

```
DROP DATABASE IF EXISTS employee;
CREATE DATABASE employee;

CREATE TABLE Departments (
  department_id   INT PRIMARY KEY,
  department_name VARCHAR(100)
);
```

```
CREATE TABLE Location (
  location_id   INT PRIMARY KEY,
  location_name VARCHAR(100)
);
```

Task 2 — Table Alteration

```
ALTER TABLE Employees ADD COLUMN email VARCHAR(100);
ALTER TABLE Employees MODIFY COLUMN Designation VARCHAR(150);
ALTER TABLE Employees DROP COLUMN Age;
ALTER TABLE Employees RENAME COLUMN Hire_date TO date_of_joining;
```

Task 3 — Table Renaming

```
RENAME TABLE Departments TO Departments_Info;
RENAME TABLE Location TO Locations;
```

Task 4 & 5 — Truncation and Dropping

```
TRUNCATE TABLE Employees;
DROP TABLE Employees;
```

5. Part B — Final Schema with Constraints

After recreating the database, the three tables were rebuilt with full constraint enforcement.

```
CREATE TABLE Departments (
  department_id   INT PRIMARY KEY,
  department_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Locations (
  location_id   INT AUTO_INCREMENT PRIMARY KEY,
  location_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Employees (
  employee_id    INT PRIMARY KEY,
  Employee_name  VARCHAR(50) NOT NULL,
  Gender         CHAR(1) CHECK (Gender IN ('M', 'F')),
  Age            INT CHECK (Age >= 18),
  Hire_date      DATE DEFAULT (CURRENT_DATE),
  Designation    VARCHAR(150),
  Salary         DECIMAL(10,2),
  email          VARCHAR(100) UNIQUE,
  department_id  INT,
  location_id    INT,
  CONSTRAINT fk_employee_department
    FOREIGN KEY (department_id) REFERENCES Departments(department_id),
  CONSTRAINT fk_employee_location
    FOREIGN KEY (location_id) REFERENCES Locations(location_id)
);
```

Constraints Used

Constraint	Purpose
PRIMARY KEY	Ensures unique identification of records.
FOREIGN KEY	Establishes relationships between tables.
NOT NULL	Prevents empty values in important fields.
UNIQUE	Ensures no duplicate values in a column.
CHECK	Restricts values based on conditions (e.g. age ≥ 18, gender M/F).
DEFAULT	Automatically assigns values (e.g. current date for Hire_date).

6. Constraint Enforcement — Proof of Validation

Three deliberate invalid inserts were run to confirm the schema rejects bad data:

Test	Attempted Insert	Rule Violated	Result
Invalid Gender	Gender = 'X'	CHECK (Gender IN ('M','F'))	Rejected
Underage Employee	Age = 15	CHECK (Age >= 18)	Rejected
Duplicate department_id	department_id = 1 (existing)	PRIMARY KEY	Rejected

Final schema verification confirmed three tables (Departments, Employees, Locations) and a fully-constrained Employees structure via SHOW TABLES and DESCRIBE Employees.

7. Sample Data & Joins

Four departments (Human Resources, Engineering, Sales, Finance), three locations (Bengaluru, Delhi, Mumbai), and eight employees were loaded for exploration.

- Join 1 — INNER JOIN: employees matched with their department and location names.
- Join 2 — LEFT JOIN: all departments returned, including any with zero employees.
- Join 3 — LEFT JOIN: all locations returned, including any with zero employees.

8. Five Analytical Queries & Results

Q1 — Headcount and average salary per department

Department	Headcount	Avg Salary
Finance	1	95,000.00
Engineering	3	67,666.67
Human Resources	2	61,000.00
Sales	2	41,000.00

Q2 — Highest-paid employee in each department

Department	Employee	Salary
Human Resources	Arjun Mehta	72,000.00
Finance	Vikram Nair	95,000.00
Sales	Rohan Gupta	42,000.00
Engineering	Karan Malhotra	70,000.00

Q3 — Employee count and total salary cost per location

Location	Employees	Total Salary Cost
Bengaluru	3	228,000.00
Delhi	3	184,000.00
Mumbai	2	90,000.00

Q4 — Gender distribution and average age by department

Department	Gender	Count	Avg Age
Engineering	F	1	29.0
Engineering	M	1	29.0
Finance	M	1	45.0

Department	Gender	Count	Avg Age
Human Resources	F	1	39.0
Human Resources	M	1	34.0
Sales	F	1	22.0
Sales	M	1	26.0

Q5 — Departments where average salary exceeds the company-wide average

Department	Avg Salary
Engineering	67,666.67
Finance	95,000.00

9. Conclusion

- The Employees Database was successfully created with proper constraints.
- Constraints ensured data integrity, accuracy, and reliability.
- Relationships between tables were established using foreign keys.
- The project highlights how SQL schema design and constraints form the backbone of relational databases.

10. Technologies Used

- MySQL / MariaDB (Database Management System)
- SQL DDL Commands (CREATE, ALTER, RENAME, TRUNCATE, DROP)
- MySQL Workbench 8.0

Author: Sharu Latha B — Student pursuing AI-driven Data Analytics | Module 3, MySQL Assignment 1